Access Paths for Efficient Ordering with Large Language Models

Fuheng Zhao[1], Jiayue Chen[2], Yiming Pan[3], Tahseen Rabbani[2], Sohaib[4]
Divyakant Agrawal[4], Amr El Abbadi[4], Paritosh Aggarwal[1], Anupam Datta[1], Dimitris Tsirogiannis[1]
{fuheng.zhao,paritosh.aggarwal,anupam.datta,d.tsirogiannis}@snowflake.com
{jiayuechen,trabbani}@uchicago.edu,yimingpan@ucla.edu,{sohaib,divyagrawal,amr}@ucsb.edu
[1]Snowflake Inc. [2]University of Chicago [3]UC Los Angeles [4]UC Santa Barbara

arXiv:2509.00303v2 [cs.DB] 3 Dec 2025

ABSTRACT

In this work, we present the `LLM ORDER BY` semantic operator as a logical abstraction and conduct a systematic study of its physical implementations. First, we propose several improvements to existing semantic sorting algorithms and introduce a semantic-aware external merge sort algorithm. Our extensive evaluation reveals that no single implementation offers universal optimality on all datasets. From our evaluations, we observe a general test-time scaling relationship between sorting cost and the ordering quality for comparison-based algorithms. Building on these insights, we design a budget-aware optimizer that utilizes heuristic rules, LLM-as-Judge evaluation, and consensus aggregation to dynamically select the near-optimal access path for LLM ORDER BY. In our extensive evaluations, our optimizer consistently achieves ranking accuracy on par with or superior to the best static methods across all benchmarks. We believe that this work provides foundational insights into the principled optimization of semantic operators essential for building robust, large-scale LLM-powered analytic systems.

PVLDB Reference Format:
Fuheng Zhao[1], Jiayue Chen[2], Yiming Pan[3], Tahseen Rabbani[2], Sohaib[4] and Divyakant Agrawal[4], Amr El Abbadi[4], Paritosh Aggarwal[1], Anupam Datta[1], Dimitris Tsirogiannis[1]. Access Paths for Efficient Ordering with Large Language Models. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/ZhaoFuheng/OrderByLLM.

1 INTRODUCTION

Recent advances in large language models (LLMs) have facilitated complex data processing, such as automatically translating natural language questions into SQL queries [15, 16, 19, 30, 73], powering autonomous problem-solving agents [22, 34, 44, 55, 57, 72, 74, 79], and introducing new declarative query languages with AI operators [2, 18, 52, 70, 71]. Building LLM declarative interface through semantic operators has also attracted significant attention from industry, with systems such as BigQuery ML [20], Snowflake AI SQL [2, 56], and Amazon Redshift's user-defined functions [5] providing support for these capabilities.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

In this work, we focus on the implementation of one specific LLM/AI semantic operator: `LLM ORDER BY`, commonly used for sorting or ranking. Sorting is a fundamental operation in data processing and underpins a wide range of analytical and decision-making tasks. `LLM ORDER BY` enables ranking items based on semantic signals, user preferences, or task-specific criteria that are difficult to capture with traditional methods. For example, in information retrieval, it is common to use LLMs to re-rank retrieved documents or passages based on the user's question or preferences [76]. In the context of relational data processing, LLMs can be leveraged to rank selected rows based on attributes that are not explicitly stored in the relational table, such as inferred sentiment or other factual information [71, 75]. Let's consider the following two concrete examples:

Example 1.1. *A User provides a query and, after an initial index filter (e.g., BM25 [49]) on a large documents corpus, the user invokes the* `LLM ORDER BY` *operator to re-rank the retrieved subset. This approach allows the LLM to generate high quality ordered result based on a nuanced semantic understanding of the user's query and preferences.*

```
SELECT p.id, p.text FROM (
    SELECT id, text FROM passages
    ORDER BY BM25(text, 'define bmt medical') DESC
    LIMIT 100 ) AS p
LLM_ORDER_BY(p.text, 'relevance to query: define bmt
     medical') DESC LIMIT 10;
```

Example 1.2. *A company wants to rank customer feedback based on specific criteria, such as product dissatisfactory or satisfactory level, and to improve its product based on the feedback. Because "dissatisfactory" or "satisfactory" is a subjective attribute, the data analyst can invoke the* `LLM ORDER BY` *operator to rank the customer feedback based on this specific criteria, requiring the LLM to highlight the subtle and nuanced levels of customer sentiment. This capability enables the company to directly address the most critical product pain points or capitalize on successful features.*

```
SELECT id, text FROM reviews
LLM_ORDER_BY(text, 'degree of positivity')
DESC LIMIT 10;
```

Given the importance of data ordering and ranking, researchers have explored a wide range of classical ideas in conjunction with LLMs. Prior works mainly study three classes of methods to implement sorting on top of LLMs: (i) pointwise sorting derives the value independently for each given key (e.g., passage relevance score, review sentiment score) [13, 51, 62, 71], (ii) pairwise sorting compares two keys and then determines the relative order between

them [36, 48, 54], and (iii) listwise sorting considers an entire set of keys simultaneously to produce a ranked order [37, 58]. There is no clear consensus on which approach is best. For example, Lotus [45], a recent LLM-powered data processing system, implements LLM-based ordering using a pairwise approach with quicksort. BigQuery ML implements LLM-based ordering using a pointwise method [28]. Furthermore, on some benchmarks, researchers find listwise sorting often provide high accuracy [37, 47], and on other benchmarks, researchers find that pairwise sorting provide higher accuracy [48].

Thus, in this paper, we aim to address the following question: **Given a semantic LLM ORDER BY task and potentially a monetary budget, which algorithm should we use to maximize sorting accuracy?** Before answering this question, we first wonder if there exists a universal optimal approach which achieves the best quality for all given tasks and datasets. To this end, we implemented existing approaches, proposed new LLM-based sorting algorithms, and conducted a systematic evaluation of different approaches within a unified experimental framework across different tasks. We observe that no algorithm outperforms the others across all tasks. In some cases, pointwise method (often criticized for its limited accuracy [48]) can, in fact, achieve strong accuracy in certain scenarios, while pairwise and listwise approaches excel in others. This highlights the importance of tailoring the ranking strategy to the task setting rather than relying on a one-size-fits-all solution. Moreover, this suggests that the `LLM Access Path` [1] for all semantic operators should be carefully selected based on query characteristics and the underlying model and data. Although this paper does not aim to optimize the selection of the access path for all operators, we present an optimizer that determines a highly accurate sorting approach for the `LLM ORDER BY the` operator.

After thoroughly examining the efficiency–effectiveness trade-offs in the initial evaluations, we conjecture a scaling relationship between compute cost and quality in semantic sorting, where increased computational consistently correlate with higher quality results. Building upon this insight, we propose two distinct optimizer configurations for the `LLM ORDER BY` operator: an **optimistic** strategy, which leverages an LLM-as-Judge to evaluate candidate quality, and a **pessimistic** strategy, which utilizes Borda count [14] aggregation to derive a robust consensus. The optimistic LLM-as-Judge optimizer relies on direct reasoning to maximize accuracy on lighter tasks (e.g., review sentiment), and the pessimistic strategy employs consensus aggregation to ensure stability and outlier mitigation in complex, long-context scenarios (e.g., passage ranking).

To conclude, our contributions can be summarized as follows:

- We propose several improvements to existing semantic sorting algorithms as well as a new semantic sorting algorithm which is an adaptation to the classical two-way merge sort algorithm [53].
- With extensive performance and quality evaluations, we demonstrate that no single existing implementation is optimal for every semantic ordering task.
- Based on empirical data, we conjecture that LLM-based sorting obeys the test-time scaling law, which relates computational expenditure to output quality, forming a foundation for an intelligent optimizer.
- We design and implement a dynamic optimizer leveraging heuristics, LLM judgment, and consensus voting to maximize ranking quality within budget constraints.
- Our extensive experiments show that this adaptive approach outperforms single static algorithms across diverse benchmarks, successfully mitigating the performance volatility often associated with fixed strategies.

This paper is structured as follows: Section 2 provides background information on semantic sorting and establishes the notation. Section 3 reviews existing algorithms and introduces our proposed extensions, including the new semantic merge sort algorithm. Section 4 details our evaluation on two benchmarks, demonstrating that no single algorithm is universally optimal and quantifying the relationship between computational budget and accuracy. Section 5 introduces an optimizer that incorporates heuristic rules, LLM guidance, and consensus aggregation. Section 6 presents extensive experiments validating the effectiveness of our proposed optimizers. Finally, Section 7 concludes with a discussion of our findings.

2 PRELIMINARY

The **LLM ORDER BY** semantic operator takes as input a list of data points along with a ranking criterion and produces an ordered list. These data points may represent diverse items, such as documents, reviews, or database rows, while the ranking criterion specifies the basis for ordering such as the relevance to a user question. Unlike traditional database settings, where the ordering attribute is explicitly stored in the table, LLM-based ordering relies on semantic judgments or inferred attributes. For example, ranking passages by relevance to a user query necessitates reasoning about semantic similarity between the query and the passage, where traditional SQL is confined to sorting by predefined attributes.

Previous LLM sorting algorithms are typically designed for passage or document ranking and are commonly categorized as pointwise, pairwise, or listwise. In this paper, we adopt a slightly different terminology. We refer to pointwise methods as **value-based**, pairwise methods as **comparison-based**, and further divide listwise methods into two variants: **external value-based** and **external comparison-based**. The intuition is that listwise methods can be viewed as batching multiple data points: either batching the derivation of values for multiple data points (external value-based) or batching the determination of the correct permutation (external comparison-based) for multiple data points. Value-based ordering methods are commonly implemented in current LLM-powered analytic systems [28, 62, 71], typically excelling on benchmarks that test factual accuracy (e.g., ranking based on masked attributes of sports players' height or celebrity' birthday). On the other hand, systems like Lotus [45] employ comparison-based ordering for semantic ordering, achieving top-tier accuracy on sentence understanding tasks.

In our work, we focus on the semantic LLM ORDER BY operator and method of implementing the operator through standard generation APIs, treating the large language model as a semantic black box. In modern data infrastructures, models, whether proprietary or managed open-source, are predominantly exposed via high-level inference interfaces to abstract away the complexities of serving,

[1] originating from classical data base systems [53]

Algorithm 1: Determine Batch Size for External Pointwise

Input: Dataset D, LLM function LLM, agreement threshold θ, maximum batch size M
$m \leftarrow 2$;
while $2m < |D|$ *and* $m < M$ **do**
  $B_1 \leftarrow D[:m]$ `// first m keys`
  $B_2 \leftarrow D[m+1:2m]$ `// next m keys`
  $B_3 \leftarrow B_1 \cup B_2$ `// combined batch of size 2m`
  Run LLM on B_1, B_2, B_3 to get V_1, V_2, V_3;
  if *any output invalid* **then**
    break;
  Merge $V_{12} = V_1 \cup V_2$;
  Compute agreement $\alpha = \#agree(V_{12}, V_3)/(2m)$;
  if $\alpha \geq \theta$ **then** $m \leftarrow 2m$;
  else return m;
return m;

Algorithm 2: Vote Routine in Quick Sort

Input: x, r_{initial}, *peers*, *peer_results*
Output: Side L or G
$L_{\text{vote}}, G_{\text{vote}} \leftarrow 0, 0$;
if $r_{initial} = l$ **then**
  $L_{\text{vote}} \leftarrow 1.5$
else
  $G_{\text{vote}} \leftarrow 1.5$
for *each* (y, r_y) *in* $(peers, peer_results)$ **do**
  if $y \in L \wedge r_y = l$ **then**
    $L_{\text{vote}} \leftarrow L_{\text{vote}} + 1$
  if $y \in G \wedge r_y = g$ **then**
    $G_{\text{vote}} \leftarrow G_{\text{vote}} + 1$
if $L_{vote} > G_{vote}$ **then return** L ;
else return G ;

hardware optimization, and security. Furthermore, emerging LLM-based analytical systems [34, 55, 60] increasingly rely on these standardized endpoints to ensure broad compatibility among different providers (e.g., OpenAI API [42], Snowflake Cortex API [56], LiteLLM [7]). Also, the monetary cost of using these models can be calculated directly using the use of input and output tokens.

3 METHODOLOGY

In this section, we review existing algorithms (e.g., standard pointwise, vanilla quicksort, and external bubble sort) and present our proposed optimizations: external pointwise, quick sort with majority voting, and semantic external merge sort.

3.1 Value-Based Ordering

In value-based ordering, the LLM assigns a score or a value to each key, and the final ranking is obtained by sorting the keys according to these values [40]. This approach is often the default implementation in many LLM-powered data systems [4, 18, 28, 71]. The value-based method requires $O(N)$ LLM function calls where N is the number of rows. Prompt Block 3.1 shows the template used to generate the relevance score for the passage ranking task.

To even further reduce the number of LLM function calls, a commonly used technique is to use the external pointwise approach, where m keys are batched into a single prompt and the model returns a list of m values. With the external pointwise approach, the number of LLM function calls becomes $O(N/m)$. In addition to reducing the number of function calls, an external pointwise method often leads to less monetary cost as well. This cost saving is achieved by sharing the instructional prefix for many keys in a batch rather than being repeated for every individual key, reducing the total number of input tokens. By amortizing the token cost of these instructions in all m items, the total number of input tokens billed is often reduced. Instead of providing a single key in Prompt 3.1, it will provide a list of keys and ask LLM to generate a list of values.

Prompt Block 1: Prompt for Value-based Passage Ranking

```
You are given a question and a passage.
Evaluate how well the passage answers the question by
     assigning a float relevance score from 0.0 to {
     max_point}, where 0.0 is not relevant and {max_point
     } is prefectly relevant.
Question: {question}
Passage: {key}
```

However, determining an appropriate batch size remains an open challenge [10]. The batch size is not only constraint by the LLM's context window but also has an effect on the model's ability to maintain consistent quality across all items in the batch. We therefore propose a simple agreement-based procedure to adaptively choose the batch size for a given task. As shown in Algorithm 1, we start with a batch of two keys and repeatedly double the batch size as long as the model's output of the individual key remains consistent between two separate batches of size m and one combined batch of size $2m$. We have added a local client-side LLM output cache. Repeated prompts with identical inputs are served directly from the cache, reducing redundant LLM function calls, and the number of LLM function calls in Algorithm 1 becomes $O(\log_2 m)$, where m is the batch size. Putting it together, we first make $O(\log_2 m)$ LLM calls to determine the appropriate batch size, and then perform $O(N/m)$ LLM calls to obtain the values for all keys.

3.2 Comparison-Based Ordering

In comparison-based ordering, the LLM is asked to directly compare two or more keys and decide the ranking order. Instead of assigning independent values to each key, the model outputs a permutation of the input keys. Compared to value-based ordering, comparison-based methods are typically more expensive, as the number of comparisons can grow quadratically with the number of keys in the worst case. Typically, researchers have implemented quick sort and bubble sort on top of the LLM comparators. For example, the external bubble sorting algorithm [58][2] sorts m keys at a time and then advances by $\frac{m}{2}$ steps. It allows the model to repeatedly compare

[2]Also referred to as the Sliding Window Strategy